# Contents



# SAFS Mesh Build Workflow

## Overview

The SAFS (San Andreas Fault System) mesh pipeline takes raw CFM (Community Fault Model) `.ts` TSurf files for one or more crustal faults and produces a single DG-ready Gmsh `.msh` (with companion `.vtu`) of a tetrahedral box mesh in which every fault is embedded as a tagged 2-D surface (`tag = 100`). The output is consumed by the SEAS DG miniapp, which requires every fault triangle to be shared by exactly two tets (`check_5`).

The pipeline is orchestrated by `run_newset_step_by_step.sh`. It builds the mesh incrementally – Step 1 starts with one fault, each subsequent step adds one more fault, and the full pipeline runs at every step. If a step fails, the offending fault is identified.

The Step-6 (all-6-faults) reference run is what typical users want. Driver flags described in Configuration.

---

## Architecture

Domain conventions

| Tag | Physical name | Meaning |
|---|---|---|
| 1 | `xm` | Box face at `x = x_min` (Dirichlet) |
| 2 | `xp` | Box face at `x = x_max` (Dirichlet) |
| 3 | `yp` | Box face at `y = y_max` (Dirichlet) |
| 4 | `ym` | Box face at `y = y_min` (Dirichlet) |
| 5 | `ztop` | Free-surface, `z = 0` (Natural) |
| 6 | `zbot` | Box bottom, `z = z_bot` (Natural) |
| 10 | `domain` | Bulk tet volume |
| 100 | `fault` | Internal interface (DG flux + RSF) |

Tag mapping is set in `safs.geo` at `Physical Surface` / `Physical Volume` lines. The pipeline assumes this mapping throughout – `FAULT_TAG = 100` is hard-coded in the Python helpers.

### Coordinate frame

Local-frame, right-handed, +z up. `v_local = v_utm − origin_utm` with the production origin at UTM Zone 11N (NAD83) `(500_000, 3_765_000, 0)` defined in `safs_origin.py`. All `.ts`, `.stl`, and `.msh`

files in the pipeline live in this local frame; UTM/local conversion is done only at the `ts_to_stl.py` boundary and verified by `validate_msh.check_9_utm_round_trip`.

Pipeline stages (textual flow)

1. `ts_to_stl.py` – one-shot CFM `.ts` to per-fault `.stl` (local frame), plus `bbox.json` and `transform.json`.
2. For every step (1..6) the driver runs the inner pipeline below:
    1. `conformalize_faults.py` cascade (per intersecting pair-group); fallback `corefine_faults` (CGAL).
    2. `dedup_coplanar_facets.py` (optional; removes CGAL autorefine self-intersection artefacts).
    3. `generate_safs_mesh.py` runs `gmsh -3 -algo3d 10` (HXT) followed by `check_mesh_tool_log.py --tool gmsh` (warning gate).
    4. `orient_fault_surface.py` and `write_fault_provenance.py`.
    5. `validate_msh.py` (raw stage = WARN-only; slivers legitimately exist before mmg3d cleans them).
    6. `mmg3d_post_pass.py` (mode `optim_relax_fault`) followed by `check_mesh_tool_log.py --tool mmg3d`, then orient + provenance + validate (warn).
    7. `mmg3d_local_patch.py` (DISABLED in production – known hole-emitter).
    8. `local_cavity_retet.py` (Steiner fan on bad cavities) followed by `check_mesh_tool_log.py --tool safs`, then orient + provenance + validate (warn).
3. Final outputs:
    - `output/newset_<N>_<faultset>/output/safs_newset_<N>_cavity.msh`
    - `output/newset_<N>_<faultset>/output/safs_newset_<N>_cavity.vtu`
    - `output/newset_<N>_<faultset>/output/fault_provenance_cavity.json`
    - `output/newset_<N>_<faultset>/output/validation_report_cavity.txt`

---

## Detailed Walkthrough

0. **`ts_to_stl.py`** – one-shot CFM conversion

Runs once when the CFM data set changes. Reads `.ts` (TSurf ASCII), applies `utm_to_local`, clamps `z > -Z0`, collapses 1 mm duplicates, drops zero-area triangles, writes `<short_name>.stl` (binary STL) plus `bbox.json` and `transform.json`. `transform.json` is the audit trail of the local-frame choice and is consumed (and verified) by every downstream stage.

- Inputs: `--in-dir CFM_data/`, `--res 1000` (resolution suffix).
- Outputs: `output/.../stl_per_fault/{short_name}.stl`, `bbox.json`, `transform.json`, `cleanup_log_<res>m.csv`.

1. Cascade conformal pairs (per step)

For each step, the driver groups faults into “intersecting pair-groups” (faults that share a polyline) and “disjoint singletons”. Pair-groups go through `conformalize_faults.py`’s cascade which:

```
cascade(faults, target_edge):
    polylines = chain(tri_tri_intersect_3d(A, B))
    for each polyline P:
        for each fault F in (A, B):
            split_parents_pierced_by(P, F)
            propagate_edge_pierces_to_neighbours(P, F)
    return manifold-conforming per-fault STLs + triangle_to_fault.json
```

Acceptance: every consecutive polyline-vertex pair is an edge in BOTH faults post-split. Manifold per fault (every edge has at most 2 triangles, no T-junctions).

If the cascade fails (`rc != 0` or `FAIL` / `manifold gate` in its log) the driver retries with the CGAL-6.1 `tools/corefine_faults` autorefine path (`conformalize_faults.py --mode cgal-corefine`). Autorefine is more robust on sharp / near-tangent intersections but emits coplanar overlapping triangle pairs that HXT later rejects – which is why `dedup_coplanar_facets.py` exists.

2. **`dedup_coplanar_facets.py`**

Removes coplanar-overlapping triangle pairs from per-fault STLs (CGAL autorefine artefact). Two triangles are flagged as overlapping if they share 2 vertex IDs and `|n_A . n_B|` is approximately 1. Algorithm: unified snap-grid vertex map across all faults, edge-to-tri index, walk pairs, drop the loser. Default ON in `run_newset_step_by_step.sh` via `ENABLE_DEDUP_COPLANAR=1`.

3. **`generate_safs_mesh.py`** – gmsh + HXT

Wraps the gmsh CLI. Reads `bbox.json` + `transform.json` + per-fault STLs; writes `safs_includes.geo` (one `Merge` line per fault), `domain_box.json`, `sizing.json`, and the resulting `safs_newset_<N>.msh`.

The `.geo` file `safs.geo` builds a `Box` of the requested extents, embeds every fault-STL surface in the box volume via `Surface{fault_surfs[]} In Volume{bulk_vol}`, attaches a `Distance + Threshold` size field (`Field[2]`: `SizeMin = 0.7 * res_f` inside `tube_radius`, ramping to `SizeMax = res_ff` over `ramp_dist`), and tags the box faces and fault surface as listed in Domain conventions.

Mesh algorithm: `Mesh.Algorithm3D = 10` (HXT). HXT is the only gmsh 3D algorithm that can recover constrained surfaces inside a volume reliably for this geometry.

Failure modes (caught by `check_mesh_tool_log.py --tool gmsh`):

- `HXT 3D mesh failed` – gmsh writes an empty volume but exits 0.
- `failed to recover constrained lines/triangles` – PLC recovery failed.
- `Found two exactly self-intersecting facets` – the dedup pass missed a pair.
- `No elements in volume 1` – empty mesh.

4. **`mmg3d_post_pass.py`** – global volume cleanup

Calls `mmg3d_O3` with the fault triangles marked `RequiredTriangles` so mmg3d can flip / collapse / Steiner-insert in the bulk WITHOUT touching the fault surface or the box faces. Mode `optim_relax_fault` (the default in the production driver) uses `-optim -opnbdy -hgradreq <hgrad>`.

The driver runs best-of-N: `MMG3D_N_TRIALS` independent restarts (random permutations of the input vertex order), each running `MMG3D_N_PASSES` sequential mmg3d invocations chained on the previous output. The trial with the largest gamma_min wins. Default `N_TRIALS=3`, `N_PASSES=3`.

In `optim_relax_fault` mode, mmg3d is allowed to insert NEW fault triangles along the polylines (relaxation of the Required constraint along fault-fault intersections). The driver therefore re-runs `write_fault_provenance.py` after this stage, using the original per-fault STLs as the KDTree source for nearest-centroid fault assignment.

5. **`mmg3d_local_patch.py`** – DISABLED in production

Sliver-targeted local mmg3d pass. Operates on the post-mmg3d output: builds a subdomain around every `gamma < gamma_thresh` tet (halo radius), tags the halo boundary as `RequiredTriangles` (tag 200), runs mmg3d on the subdomain, stitches the result back. Default OFF (`ENABLE_MMG3D_LOCAL_PATCH=0`) – empirically introduces surface holes through the stitch step (R-006 producer-side assertion now halts the pipeline if it does), and produces locally non-orientable fault-sheet topology that even the dihedral-aware orient cannot repair. To be rewritten.

6. **`local_cavity_retet.py`** – final gamma_min lift

For each remaining `gamma < LOCAL_CAVITY_GAMMA_THRESH` tet T:

```
cavity   = T + face-adjacent neighbours T1..T4    (at most 5 tets)
boundary = union of cavity-tet faces
           minus faces shared between cavity tets
S        = mean of all cavity vertices             (Steiner point)
out      = { (a, b, c, S) for (a, b, c) in boundary }   (Steiner fan)
```

If gamma_min of the new fan is no better than gamma_min of the old cavity, the fan is reverted (no change). Cross-fault conformity is preserved: every cavity-boundary triangle remains a face of exactly one new tet (the Steiner-fan tet), so fault triangles still satisfy `check_5`. Lift on the Step-6 production mesh: gamma_min `2.08e-06` to `6.51e-06`.

7. **`orient_fault_surface.py`** – winding canonicalizer (R-003)

Runs after every mesh-mutating stage. BFS-propagates fault-tri winding within each manifold-connected sub-sheet, with a dihedral-aware pair table at non-manifold (count >= 3) edges:

- `count == 1` (perimeter): no propagation.
- `count == 2` (manifold): propagate to the unique neighbour.
- `count >= 3`: pair tris by anti-parallel third-vertex bearings (dihedral near 180 degrees, so smooth manifold continuation across the X- or T-junction); propagate within each pair only; tris without an anti-parallel partner are terminators (no propagation through this edge).

Without this rule, a naive BFS through all edges either over-propagates (forces two crossing fault sheets into the same winding, so ParaView back-face gaps appear in one) or under-propagates (skips legitimate within-pair flips at X-junctions).

8. **`write_fault_provenance.py`** – per-fault triangle attribution

After every mmg3d / cavity stage that may add fault triangles, this script re-attributes every `tag == 100` tri to its CFM source fault by KDTree-nearest-centroid match against the original per-fault STLs. Output: `fault_provenance.json` with `faults[<short_name>].triangle_indices_in_msh`.

This file is consumed by `check_8_provenance_partition` (validate_msh) and is the source for the `fault_id` cell-data array used in ParaView coloring (see Visualizing the output).

9. **`validate_msh.py`** – 13-check gate

Each stage of the pipeline runs validate_msh after orient + provenance.

| # | Check | What it asserts |
|---|---|---|
| 1 | tag_inventory | All expected tags present |
| 2 | box_arithmetic | Bulk-volume extents match expected |
| 3 | fault_clearance | Every fault is at least 5 km from any box face |
| 4 | freesurface_trace | Fault-surface intersections at z=0 are clean |
| 5 | internal_interface | Every tag-100 tri has 2 incident tets |
| 6 | fault_edge_length | Fault tri edges roughly equal res_f |
| 7 | far_field_edge | Far-field tet edges roughly equal res_ff |
| 8 | provenance_partition | Provenance partitions tag-100 cleanly |
| 9 | utm_round_trip | Sampled vertices land in CFM bbox |
| 10 | tet_quality | gamma_min, gamma_mean, sliver fraction, INVERTED-TET count, R-402 near-fault sliver |
| 11 | tube_uniformity | In-tube tet edge roughly equal res_f |

| # | Check | What it asserts |
|---|---|---|
| 12 | surface_closure | Every 1-tet bdry face has a tagged tri (R-001) |
| 13 | fault_orientation | Winding consistent (dihedral-aware) (R-002) |

The driver wires validate as WARN-only at every stage in production: the R-402 near-fault sliver gate is a quality metric, not a topology defect, and many SAFS X-junctions intrinsically produce sub-cell tets there. Real topology defects (5/12/13 + the inverted-tet hard-fail in 10) still print as `[FAIL]` lines into the per-stage `validate_<stage>.log`; users inspect those manually if a downstream solver behaves badly.

10. **`check_mesh_tool_log.py`** – stderr-warning gate

Defense-in-depth against tools that exit 0 despite emitting fatal warnings. Runs after every `gmsh` / `mmg3d_O3` / `local_cavity_retet` invocation and greps the captured log for a curated set of fatal patterns (see `check_mesh_tool_log.py` `_GMSH_PATTERNS`, `_MMG3D_PATTERNS`, `_SAFS_PATTERNS`). Exits 1 if any pattern matches; the driver's `if ! ... ; then return N; fi` envelope aborts the pipeline.

11. **`convert_msh.py`** – **`.msh`** to **`.vtu`** (and Dolfin XML)

Last-mile converter. Reads the cavity `.msh` and writes:

- `safs_newset_<N>_cavity.vtu` – ParaView UnstructuredGrid with the `physical` cell-data array.
- `safs_newset_<N>_cavity.xml` – volume-only Dolfin XML.
- `safs_newset_<N>_cavity_facet_region.xml` – boundary-tag side-car.

The driver does NOT call `convert_msh.py` directly; convert + per-fault labelling is done as a separate post-step (see Visualizing the output).

---

## Call graph (Step-6 critical path)

```
run_newset_step_by_step.sh (driver)
  run_step 6
    conformalize_faults.py --mode cascade   (per pair-group)
      corefine_faults (CGAL, fallback)
    dedup_coplanar_facets.py
    generate_safs_mesh.py
      gmsh -3 safs.geo -algo3d 10           (HXT)
      check_mesh_tool_log.py --tool gmsh
    write_fault_provenance.py               (raw stage)
    orient_fault_surface.py                 (raw stage)
    validate_msh.py                         (raw stage; warn-only)
    mmg3d_post_pass.py
      mmg3d_O3 (best-of-N_TRIALS x N_PASSES)
      check_mesh_tool_log.py --tool mmg3d
    write_fault_provenance.py               (mmg3d stage)
    orient_fault_surface.py                 (mmg3d stage)
    validate_msh.py                         (mmg3d stage; warn)
    local_cavity_retet.py
      check_mesh_tool_log.py --tool safs
    write_fault_provenance.py               (cavity stage)
    orient_fault_surface.py                 (cavity stage)
    validate_msh.py                         (cavity stage; warn)
```

`mmg3d_local_patch.py` is wired in but disabled. Re-enable with `ENABLE_MMG3D_LOCAL_PATCH=1` only when you know the patch stage has been fixed.

---

## Configuration (environment variables of the driver)

| Variable | Default | What it does |
|---|---|---|
| `START_FROM` | 1 | Skip earlier steps; 6 = run only Step 6 |
| `ENABLE_DEDUP_COPLANAR` | 0 | Run `dedup_coplanar_facets.py` (set 1) |
| `ENABLE_MMG3D_POSTPASS` | 0 | Run `mmg3d_post_pass.py` (set 1) |
| `MMG3D_MODE` | optim | Production: `optim_relax_fault` |
| `MMG3D_HMIN` | 100 | Passed to mmg3d |
| `MMG3D_HMAX` | 25000 | Passed to mmg3d |
| `MMG3D_HGRAD` | 1.3 | Passed to mmg3d |
| `MMG3D_HAUSD` | 50 | Passed to mmg3d |
| `MMG3D_N_PASSES` | 3 | mmg3d sequential passes per trial |
| `MMG3D_N_TRIALS` | 3 | Best-of-N restarts |
| `ENABLE_MMG3D_LOCAL_PATCH` | 0 | Keep at 0 until the patch is fixed |
| `ENABLE_LOCAL_CAVITY_RETET` | 0 | Run `local_cavity_retet.py` (set 1) |
| `LOCAL_CAVITY_GAMMA_THRESH` | 1e-3 | Cavity-retet gamma threshold |
| `LOCAL_CAVITY_REVERT_TOL` | 0.0 | Revert if new fan no better by this much |

Production Step-6 invocation (the one that produced the current cavity VTU):

```
conda run -n pythonenv bash -c "\
  ENABLE_DEDUP_COPLANAR=1 \
  ENABLE_MMG3D_POSTPASS=1 MMG3D_MODE=optim_relax_fault \
  MMG3D_HAUSD=50 MMG3D_N_PASSES=3 MMG3D_N_TRIALS=3 \
  ENABLE_MMG3D_LOCAL_PATCH=0 \
  ENABLE_LOCAL_CAVITY_RETET=1 LOCAL_CAVITY_GAMMA_THRESH=1e-3 \
  START_FROM=6 \
  bash miniapps/seas/safs/mesh/run_newset_step_by_step.sh"
```

---

## Current production mesh

Reference Step-6 cavity output (built 2026-05-02 with all guards in place):

`mesh/output/newset_6_all6_garnetfirst/output/safs_newset_6_cavity.msh`
`mesh/output/newset_6_all6_garnetfirst/output/safs_newset_6_cavity.vtu`

Quality metrics from `validate_msh.py`:

| Metric | Value |
|---|---|
| `gamma_min` | 6.51e-06 |
| `gamma_mean` | 0.850 |
| `min_edge_m` (smallest tet edge) | 0.20 m |
| Mean fault tri edge | 852 m |
| Max fault tri edge | 2528 m |
| Target `res_f` | 1000 m |
| Target `res_ff` | 20000 m |

| Metric | Value |
|---|---|
| Mean in-tube tet edge | 958 m |
| p95 in-tube tet edge | 1353 m |
| Frac. in-tube edges within +/-25% | 74% |
| n_tets | 1,099,696 |
| Near-fault slivers (gamma < 0.05) | 267 (0.024%) |
| check_5 / check_12 / check_13 | all PASS |

The single failing check is check_10 R-402 near-fault sliver gate (quality metric, not a topology defect; intrinsic to SAFS X-junction geometry).

---

## Visualizing the output

After a successful run, label per-fault and emit a final VTU:

```
conda run -n pythonenv python miniapps/seas/safs/mesh/convert_msh.py \
    --msh output/newset_6_<dir>/output/safs_newset_6_cavity.msh \
    --vtu output/newset_6_<dir>/output/safs_newset_6_cavity.vtu \
    --no-xml

conda run -n pythonenv python /tmp/label_per_fault.py \
    output/newset_6_<dir>/output/safs_newset_6_cavity.msh \
    output/newset_6_<dir>/output/fault_provenance_cavity.json \
    output/newset_6_<dir>/output/safs_newset_6_cavity.vtu
```

The final VTU carries three cell-data arrays:

| Array | Meaning |
|---|---|
| physical | gmsh tag (1-6 box, 100 fault, 10 bulk) |
| fault_id | 1..6 per CFM fault (mapping printed by labeller) |
| fault_component | 1..K manifold-connected sub-sheet (dihedral) |

In ParaView: Threshold on physical == 100 to isolate the fault, then color by fault_id (categorical, range 0.5 to 6.5) to see each CFM fault in its own color.

---

## Conventions

- Error handling. Each Python helper exits non-zero on internal failure; the bash driver's if ! ... ; then return N; fi envelope aborts. Logs always go through > $OUTDIR/<tool>.log 2>&1. The check_mesh_tool_log.py gate is the second layer for tools that exit 0 on fatal warnings.
- Memory and I/O. Everything reads/writes .msh (gmsh22 ASCII) via meshio. Large meshes are kept on disk between stages; in-memory state is per-tool only. There is no shared in-process pipeline daemon.
- Parallelism. Single-process Python plus single-process gmsh / mmg3d_O3 (these tools are internally threaded but the SAFS scripts do not parallelize across faults). MPI is NOT used in the mesh build, only in the SEAS solver downstream.

- Naming. Per-fault meshes use the FAULT_SHORT_NAMES mapping in safs_origin.py. Output directories follow output/newset_<step>_<faultset>/. Pipeline stages within a step write next to each other in output/<step>/output/safs_newset_<step>_<stage>.msh (raw / mmg3d / patch / cavity).
- Configuration. All tunables are environment variables on the bash driver (see Configuration) or --flag arguments on the Python helpers. Numerical constants are NEVER hardcoded inside the Python helpers; they come from bbox.json, transform.json, domain_box.json, or sizing.json so the pipeline can be re-run on a different region without code changes.

---

## Gotchas

1. **fault_tag = 100** is hard-coded. Every Python helper assumes tag 100 is the fault. If you change safs.geo's Physical Surface("fault", 100) line, you also need to change FAULT_TAG = 100 in validate_msh.py, orient_fault_surface.py, write_fault_provenance.py, mmg3d_post_pass.py, mmg3d_local_patch.py, local_cavity_retet.py.
2. Validate is WARN-only at every production stage. The driver intentionally swallows validate's non-zero exit. Topology-fatal regressions print [FAIL] to validate_<stage>.log but do NOT halt the pipeline. The hard gates are: (a) tool-exit code, (b) check_mesh_tool_log.py, (c) producer-side R-006 assertions inside mmg3d_local_patch.py. If you notice the cavity output looks wrong, tail -50 output/.../validate_cavity.log first.
3. Inverted tets are still hard-fail inside **check_10**, even though the surrounding validate envelope is warn-only – the validate program exits 1 in either case, but the pipeline only reports the FAIL message and moves on. To turn inverted-tet detection back into a pipeline abort, replace one of the four if ! python validate_msh.py ...; then echo soft-fail; fi blocks in the driver with a strict envelope (or add a separate grep -q "inverted tet" ... after validate).
4. **mmg3d_local_patch** is OFF for a reason. It produces meshes that look gamma_min-better but have surface holes (R-001 check_12 catches them now) and locally non-orientable fault sheets that no orient pass can repair (the patch leftover from the last audit was 14 manifold-edge plus 1 count-3 inconsistencies even after the dihedral-aware orient). Re-enable only after rewriting _stitch_back.
5. gmsh-HXT is non-deterministic. Successive runs of generate_safs_mesh.py on the same dedup'd STL can land on slightly different triangulations; rare HXT 3D failures (Found two exactly self-intersecting facets) appear stochastically. The check_mesh_tool_log.py --tool gmsh gate catches the failure immediately even when gmsh exits 0 with an empty volume; the user then re-runs the cascade with a different autorefine seed.
6. Soft **validate** after raw HXT is intentional. The raw HXT mesh WILL have ~300 near-fault slivers; the R-402 quality gate would abort every single end-to-end run if it were strict. The downstream mmg3d_post_pass plus local_cavity_retet repair them.
7. **orient_fault_surface.py** runs BEFORE **validate_msh** at every stage. check_13's BFS uses the same dihedral-aware logic as orient, so check_13 will report 0 winding flips even on a buggy input – the guarantee is that orient_fault_surface.py produced the canonical winding AND check_13 confirms it, not that check_13 alone catches upstream defects. If you bypass orient (e.g. validate a hand-crafted .msh), check_13 may falsely report 0 flips when there are real defects within manifold sub-sheets.
8. **write_fault_provenance.py** after **optim_relax_fault** mmg3d. mmg3d in optim_relax_fault mode is allowed to add NEW fault triangles along intersection polylines. The KDTree-nearest-centroid attribution re-runs against the ORIGINAL per-fault STLs so the new triangles inherit a sensible CFM source. If you rebuild provenance using a downstream .msh as the KDTree source instead of the original STL, nearest-centroid matching breaks at the junctions.

## Open questions

- **`mmg3d_local_patch`** rewrite path. The current `_stitch_back` is the surface-hole producer. A principled fix is non-trivial because the patch step legitimately deletes interior tets and re-inserts a different triangulation that does not align with the surrounding mesh's vertex set. Best path is probably to drop the patch step entirely and tighten `mmg3d_post_pass`'s sliver targeting via a smaller `hmin` and more trials.
- Near-fault sliver eradication. Even after cavity-retet, ~270 of 1.1 M tets sit at gamma below 0.05 within 5 km of the fault. These are at the fault-fault X-junctions where the polyline geometry forces the triangulation locally. Whether these slivers actually destabilize SEAS DG flux assembly is unverified – the R-402 gate is conservative.
- Pipeline-as-graph. The driver is a flat bash script with explicit `if ! ... ; then return N; fi` chains. A small make / nextflow / snakemake port would make the dependency structure first-class but would add another tool to the project; not worth it until someone needs to run multiple step-6 builds in parallel.